

EXPERIMENT 2

Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

What is Multiple & Regularized Regression?

Regression is a supervised machine learning technique used to predict continuous output values.

In this experiment, we implement

:

- Multiple Linear Regression → Uses multiple input features
- Ridge Regression → Linear Regression + L2 Regularization
- Lasso Regression → Linear Regression + L1 Regularization

These techniques are used when:

- Dataset has multiple features
- Risk of overfitting exists

1.Dataset Source (Kaggle)

Dataset Name :

House Price Regression Dataset

Dataset Link :

<https://www.kaggle.com/datasets/prokshitha/home-value-insights>

Target Variable : SalePrice (House price)

Type : Continuous numerical value → Regression Problem

Use :

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression

2. Dataset Description

The Home Value Insights dataset contains information about residential properties and their prices. Each row represents one house and multiple features that influence its value.

Input (Independent) Features :

Feature	Meaning
Square_Footage	Total area of the house in square feet
Num_Bedrooms	Number of bedrooms
Num_Bathrooms	Number of bathrooms
Year_Built	Year the house was constructed
Lot_Size	Size of land lot in acres
Garage_Size	Number of cars the garage can fit
Neighborhood_Quality	Quality rating of the neighborhood (1–10)

These features influence the house value — larger size, more rooms, and better neighborhood often increase price.

Target (Dependent) Variable :

House_Price

- Continuous numerical value
- Represents the selling price of the house

Because the target is continuous, regression models are appropriate.

3. Mathematical Formulation of the Algorithms

Multiple Linear Regression

Multiple regression predicts an outcome using several features:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Where:

- y = Predicted house price
- $x_1, x_2, \dots, x_n$ = Input features
- β_0 = Intercept
- $\beta_1, \beta_2, \dots, \beta_n$ = Coefficients (weights)

The model tries to minimize the Mean Squared Error (MSE) between actual and predicted values.

Ridge Regression (L2 Regularization)

Ridge adds a penalty on coefficient magnitudes:

$$Loss = MSE + \lambda \sum \beta_i^2$$

- λ (lambda) controls strength of regularization
- Larger $\lambda \rightarrow$ more shrinkage

Ridge reduces overfitting by preventing very large coefficients.

Lasso Regression (L1 Regularization)

Lasso also regularizes but with absolute values
:

$$Loss = MSE + \lambda \sum |\beta_i|$$

- Can shrink some coefficients to zero
- Effectively selects features

Lasso is useful when many features might be irrelevant.

4. Algorithm Limitations**Multiple Linear Regression :**

- Assumes a linear relationship between inputs and output
- Sensitive to outliers
- Struggles with multicollinearity (features correlated with each other)

Ridge Regression :

- Reduces overfitting but does not remove unimportant features
- Requires selecting a good λ value

Lasso Regression :

- Can eliminate too many features if λ is too large
- Performance may drop if correlated features are present

5. Methodology / Workflow

Step-by-Step Workflow :

1. **Dataset Loading**
 - Read CSV downloaded from Kaggle
2. **Data Preprocessing**
 - Handle missing values
 - Select numerical input features
 - Scale features if required
3. **Train-Test Split**
 - Divide dataset (e.g., 80% training, 20% testing)
4. **Model Training**
 - Train Multiple Linear Regression
 - Train Ridge Regression
 - Train Lasso Regression
5. **Prediction**
 - Predict house prices on test set
6. **Evaluation**
 - Calculate metrics like MAE, MSE, RMSE, R^2
7. **Hyperparameter Tuning**
 - Tune λ (alpha) values for Ridge and Lasso using cross-validation

6. Performance Analysis

To analyze model performance, we use :

Evaluation Metrics :

Metric	Meaning
MAE	Average absolute error between predicted and actual
MSE	Average squared prediction error
RMSE	Root of MSE (same units as target)
R ² Score	Proportion of variance explained by the model

Interpretation:

- Smaller errors (MAE, MSE, RMSE) → better prediction
- Higher R² → stronger model fit

For Ridge and Lasso, good tuning often improves performance over plain linear regression.

7. Hyperparameter Tuning

Ridge & Lasso Hyperparameter

- Alpha (λ) determines regularization strength

Using GridSearchCV, we automatically test multiple alpha values and choose the best according to the cross-validated score.

Why Tune?

- Controls bias–variance trade-off
- Helps find a model that generalizes better on unseen data

Using the House Value Insights dataset from Kaggle, we build three regression models to predict house prices. Multiple regression provides a baseline. Ridge reduces overfitting using L2 regularization, while Lasso adds feature selection through L1 regularization. Performance is evaluated with standard regression metrics, and hyperparameter tuning is done with cross-validation.

CODE :

```
# =====
# IMPORT LIBRARIES
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# LOAD DATASET
# =====
# Upload home_value_insights.csv before running

df = pd.read_csv("/content/sample_data/house_price_regression_dataset.csv")
print("Dataset Preview:\n", df.head())

# =====
# DATA CHECK & CLEANING
# =====
print("\nDataset Info:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

df = df.dropna()

# =====
# FEATURE & TARGET SPLIT
# =====
X = df.drop(columns=["House_Price"])
y = df["House_Price"]

# =====
# FEATURE SCALING
# =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

# =====
# TRAIN-TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# =====
# MULTIPLE LINEAR REGRESSION
# =====

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

print("\n===== MULTIPLE LINEAR REGRESSION =====")
print("MAE:", mean_absolute_error(y_test, y_pred_lr))
print("MSE:", mean_squared_error(y_test, y_pred_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
print("R2 Score:", r2_score(y_test, y_pred_lr))

plt.figure()
plt.scatter(y_test, y_pred_lr)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Multiple Linear Regression")
plt.show()

# =====
# RIDGE REGRESSION
# =====

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

print("\n===== RIDGE REGRESSION =====")

```

```

print("MAE:", mean_absolute_error(y_test, y_pred_ridge))
print("MSE:", mean_squared_error(y_test, y_pred_ridge))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_ridge)))
print("R2 Score:", r2_score(y_test, y_pred_ridge))

```

```

plt.figure()
plt.scatter(y_test, y_pred_ridge)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Ridge Regression")
plt.show()

```

```

# =====

```

```

# LASSO REGRESSION

```

```

# =====

```

```

lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

```

```

print("\n===== LASSO REGRESSION =====")
print("MAE:", mean_absolute_error(y_test, y_pred_lasso))
print("MSE:", mean_squared_error(y_test, y_pred_lasso))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lasso)))
print("R2 Score:", r2_score(y_test, y_pred_lasso))

```

```

plt.figure()
plt.scatter(y_test, y_pred_lasso)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Lasso Regression")
plt.show()

```

```

# =====

```

```

# MODEL COMPARISON GRAPH

```

```

# =====

```

```

models = ["Linear", "Ridge", "Lasso"]
r2_scores = [

```



```

r2_score(y_test, y_pred_lr),
r2_score(y_test, y_pred_ridge),
r2_score(y_test, y_pred_lasso)
]

plt.figure()
plt.bar(models, r2_scores)
plt.ylabel("R2 Score")
plt.title("Regression Model Comparison")
plt.show()

# =====
# HYPERPARAMETER TUNING
# =====

param_grid = {"alpha": [0.001, 0.01, 0.1, 1, 10, 100]}

ridge_grid = GridSearchCV(Ridge(), param_grid, cv=5)
ridge_grid.fit(X_train, y_train)

lasso_grid = GridSearchCV(Lasso(), param_grid, cv=5)
lasso_grid.fit(X_train, y_train)

print("\n===== HYPERPARAMETER TUNING RESULTS =====")
print("Best Ridge Alpha:", ridge_grid.best_params_)
print("Best Lasso Alpha:", lasso_grid.best_params_)

```

OUTPUT :

Dataset Preview :

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size \
0	1360	2	1	1981	0.599637
1	4272	3	3	2016	4.753014
2	3592	1	2	2016	3.634823
3	966	1	2	1977	2.730667
4	4926	2	1	1993	4.699073

	Garage_Size	Neighborhood_Quality	House_Price
0	0	5	2.623829e+05
1	1	6	9.852609e+05
2	0	9	7.779774e+05

```

3      1      8 2.296989e+05
4      0      8 1.041741e+06

```

Dataset Info:

RangeIndex: 1000 entries, 0 to 999

Data columns (total 8 columns):

```

#  Column      Non-Null Count  Dtype
---  -
0  Square_Footage    1000 non-null  int64
1  Num_Bedrooms      1000 non-null  int64
2  Num_Bathrooms     1000 non-null  int64
3  Year_Built        1000 non-null  int64
4  Lot_Size          1000 non-null  float64
5  Garage_Size       1000 non-null  int64
6  Neighborhood_Quality 1000 non-null  int64
7  House_Price       1000 non-null  float64

```

dtypes: float64(2), int64(6)

memory usage: 62.6 KB

None

Missing Values:

```

Square_Footage    0
Num_Bedrooms      0
Num_Bathrooms     0
Year_Built        0
Lot_Size          0
Garage_Size       0
Neighborhood_Quality 0
House_Price       0
dtype: int64

```

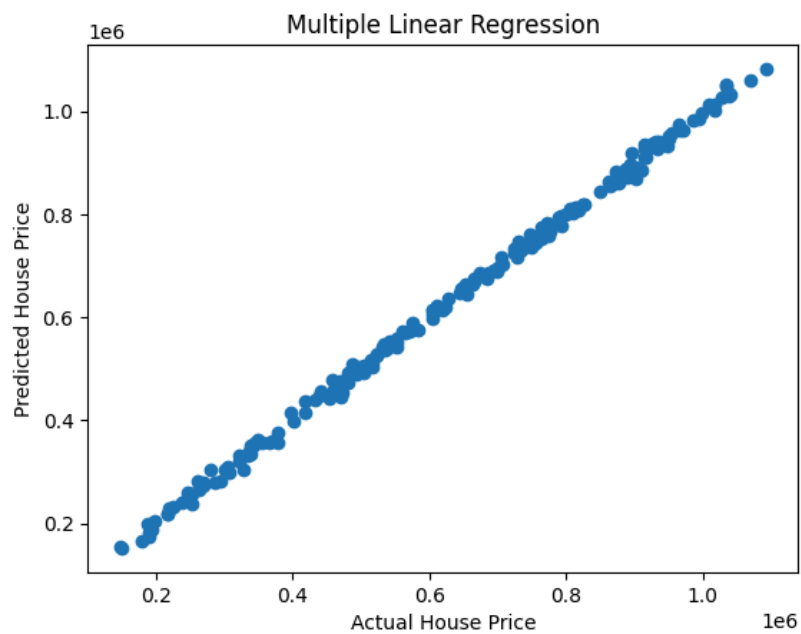
===== MULTIPLE LINEAR REGRESSION =====

MAE: 8174.583600006606

MSE: 101434798.50563568

RMSE: 10071.484424137074

R2 Score: 0.9984263636823413



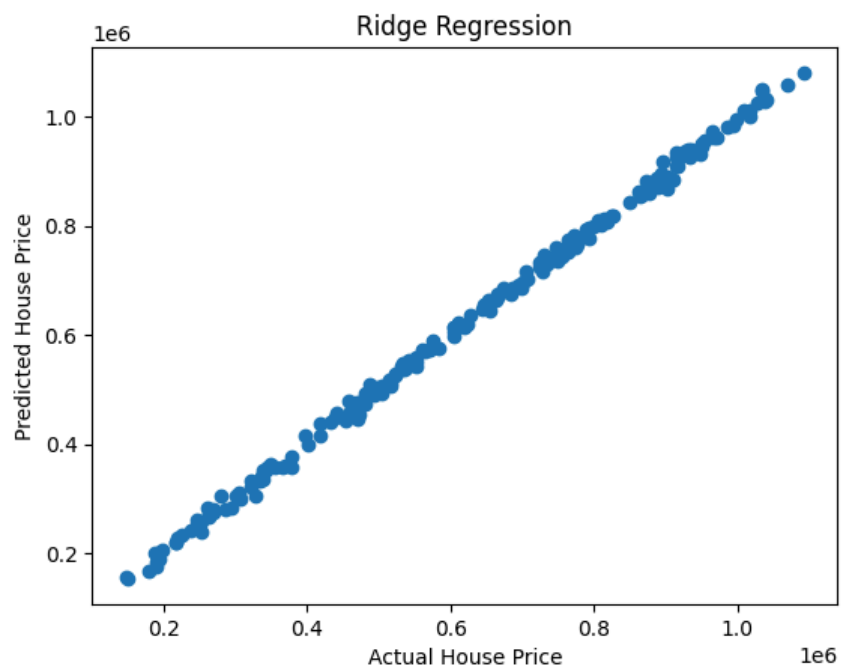
===== RIDGE REGRESSION =====

MAE: 8241.898358133865

MSE: 102486578.62155962

RMSE: 10123.565509323265

R2 Score: 0.998410046605628



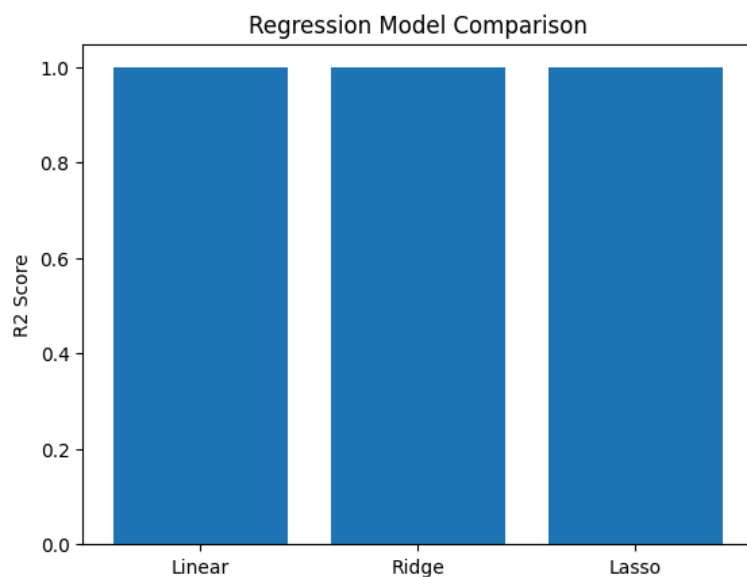
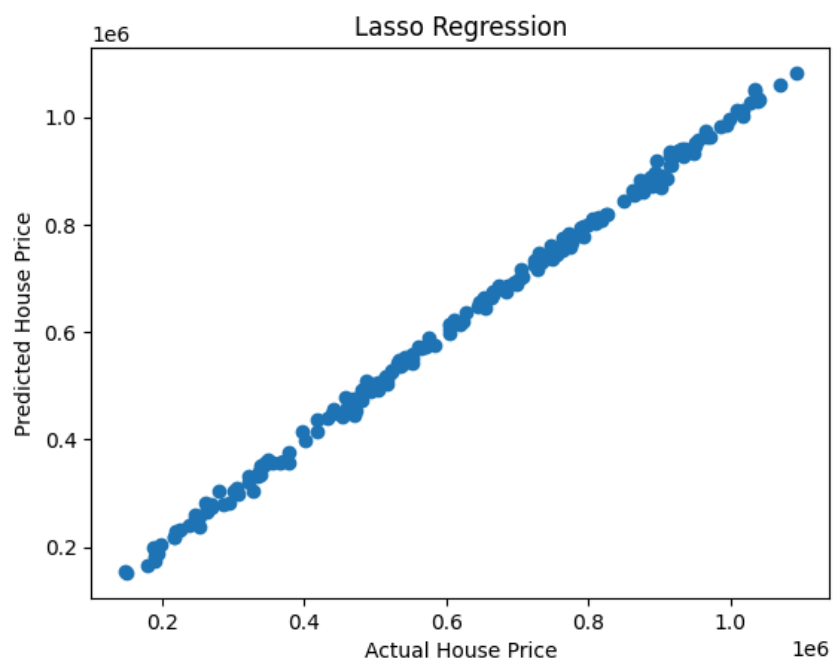
===== LASSO REGRESSION =====

MAE: 8174.585809784338

MSE: 101434824.93285091

RMSE: 10071.48573611912

R2 Score: 0.9984263632723556



===== HYPERPARAMETER TUNING RESULTS =====

Best Ridge Alpha: {'alpha': 0.001}

Best Lasso Alpha: {'alpha': 10}